Universidad San Carlos de Guatemala

Teoría de Sistemas 2

Ing. Pedro Domingo

MANUAL TÉCNICO

Carlos Enrique Lopez Lopez 201831102

Kenny Marco Augusto Salazar 202031554

Mario Raúl Yancor Ocaña 201930761

Quetzaltenango, 27 de abril de 2026

ÍNDICE

1. Introducción **3**
2. Herramientas de desarrollo **3**
3. Arquitectura N-Capas **3**
4. Estructura del proyecto **4**
5. Modelo de base de datos **5**
Relaciones principales **5**
6. Módulos principales del sistema **6**
Reglas de negocio destacadas **6**
7. Endpoints de la API **7**
Cuerpos de petición principales **10**
8. Validaciones, seguridad e integridad **11**
9. Instalación y despliegue **11**
Requisitos previos **11**
Configuración de base de datos **11**
Variables .env del backend **11**
Ejecución del backend **12**
Ejecución del frontend **12**
10. Responsividad y Pruebas en Dispositivos Móviles **13**
11. Diagramas del sistema **17**

1. Introducción

IngeniaMath es una plataforma web de aprendizaje adaptativo orientada a aspirantes de Ingeniería USAC. El sistema permite realizar diagnóstico inicial, generar rutas de aprendizaje, administrar ejercicios, ejecutar sesiones de práctica, realizar simulacros de examen y gestionar usuarios por roles.

Este manual técnico describe la estructura interna del proyecto, las tecnologías utilizadas, la base de datos, las reglas de negocio principales, los endpoints disponibles y las instrucciones de instalación para desarrollo y pruebas.

2. Herramientas de desarrollo

Área	Tecnología
Backend	PHP 8.2 o superior, Laravel 12, Laravel Sanctum
Frontend	React 19, Vite, React Router DOM, Axios
Base de datos	PostgreSQL, base ingeniaMathDB, esquema math
Renderizado matemático	KaTeX, react-katex y better-react-mathjax para expresiones matemáticas
Seguridad	Laravel Sanctum para tokens, password_hash/Bcrypt para contraseñas, middleware de roles
Control de versiones	Git y GitHub con ramas main, develop, feature y fix
Sistema operativo de desarrollo	Windows y Linux, con soporte para ejecución local por red WiFi

3. Arquitectura N-Capas

El proyecto está organizado siguiendo una arquitectura N-Capas. Cada capa tiene una responsabilidad definida para mantener el código ordenado, facilitar mantenimiento y permitir que varios integrantes trabajen módulos distintos sin mezclar responsabilidades.

Capa	Responsabilidad
Presentación	Frontend React. Contiene páginas, formularios, componentes visuales, navegación y servicios Axios.
Controladores	Laravel API Controllers. Reciben peticiones HTTP, validan entrada y delegan al service correspondiente.
Lógica de negocio	Services. Aplican reglas del sistema: autenticación, diagnóstico, rutas, práctica, simulacros y flujo de ejercicios.

Acceso a datos	DAO y Models. Ejecutan consultas SQL, operaciones sobre PostgreSQL y recuperación de entidades.
Persistencia	Base PostgreSQL con esquema math y tablas normalizadas para usuarios, ejercicios, práctica, diagnóstico y simulacros.

4. Estructura del proyecto

La estructura principal del repositorio se divide en backend, frontend y scripts de base de datos.

```
IngeniaMath/
├── backend/
│   ├── app/
│   │   ├── DAO/
│   │   ├── Http/Controllers/Api/
│   │   ├── Http/Middleware/
│   │   ├── Http/Requests/
│   │   ├── Models/
│   │   └── Services/
│   ├── config/
│   ├── routes/api.php
│   └── composer.json
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── context/
│   │   ├── pages/
│   │   ├── router/
│   │   └── services/
│   └── package.json
└── DB/
    ├── script_v2.sql
    └── inserts_completos_v2.sql
```

Los controladores principales se encuentran en backend/app/Http/Controllers/Api. La lógica del sistema está en backend/app/Services y las consultas específicas se concentran en backend/app/DAO.

5. Modelo de base de datos

La base de datos se crea con el script DB/script_v2.sql. Se utiliza PostgreSQL con el esquema math. El script define roles, secuencias, tablas, llaves primarias, llaves foráneas, restricciones CHECK, índices y permisos para el usuario adminMath.

Grupo	Tablas principales
Usuarios y seguridad	roles, usuarios, perfiles_estudiante, tutor_estudiante, tokens_recuperacion_password, auditoria_actividad
Contenido académico	modulos_tematicos, subtemas, prerrequisitos_subtema, ejercicios, opciones_ejercicio, ejercicios_relacionados, revisiones_ejercicio
Diagnóstico y ruta	tests_diagnostico, test_diagnostico_preguntas, intentos_diagnostico, respuestas_diagnostico, resultados_diagnostico_modulo, rutas_aprendizaje, ruta_aprendizaje_detalle
Práctica	sesiones_practica, respuestas_practica, ejercicios_guardados
Simulacros	configuraciones_simulacro, configuracion_simulacro_modulo, simulacros, simulacro_preguntas, simulacro_resultados_modulo
Recursos y foro	recursos_educativos, flashcards, recurso_por_ejercicio, hilos_foro, respuestas_foro
Plan semanal	planes_estudio_semanal, plan_estudio_dia

Relaciones principales

Campo FK	Referencia	Descripción
usuarios.rol_id	roles.id	Cada usuario pertenece a un rol.
subtemas.modulo_id	modulos_tematicos.id	Cada subtema pertenece a un módulo temático.
ejercicios.modulo_id / subtema_id	modulos_tematicos.id / subtemas.id	Cada ejercicio queda clasificado por módulo y subtema.
opciones_ejercicio.ejercicio_id	ejercicios.id	Cada ejercicio de opción múltiple tiene opciones asociadas.
sesiones_practica.estudiante_id	usuarios.id	Cada práctica pertenece a un estudiante.
respuestas_practica.sesion_practica_id	sesiones_practica.id	Cada respuesta se guarda dentro de una sesión.

simulacros.configuracion_simulacro_id	configuraciones_simulacro.id	Cada simulacro utiliza la configuración activa al momento de iniciarlo.
simulacro_preguntas.simulacro_id	simulacros.id	Cada simulacro tiene preguntas seleccionadas aleatoriamente.

6. Módulos principales del sistema

Módulo	Función
Autenticación y usuarios	Registro, login, logout, perfil, administración de usuarios y control por roles.
Banco de ejercicios	Creación, edición, revisión, aprobación, publicación y deshabilitación de ejercicios.
Diagnóstico inicial	Test obligatorio para estudiantes, cálculo de resultados por módulo y clasificación de dominio.
Ruta y plan de estudio	Ruta personalizada por debilidades y generación de plan semanal según horas disponibles.
Práctica y motor de recomendación	Práctica libre o guiada, selección de ejercicios según historial, ajuste de dificultad, guardados e historial.
Simulacros de examen	Simulacro cronometrado, configuración global por administrador, resultados por módulo e historial.
Perfil	Actualización de datos personales, foto, biografía, teléfono y contraseña.

Reglas de negocio destacadas

- El diagnóstico clasifica cada módulo como DOMINADO, EN_DESARROLLO o DEFICIENTE.
- La ruta de aprendizaje prioriza módulos con menor rendimiento y agrega prerrequisitos cuando corresponde.
- El banco de ejercicios maneja el flujo BORRADOR, EN_REVISION, APROBADO, PUBLICADO y DESHABILITADO.
- La práctica guiada prioriza módulos y subtemas con menor porcentaje de aciertos del estudiante.
- La práctica evita repetir ejercicios respondidos correctamente durante los últimos 7 días.
- El nivel de dificultad sube después de 3 aciertos consecutivos y baja si el estudiante falla más del 60 por ciento.
- Los simulacros utilizan duración de 90 minutos por defecto, pero el administrador puede configurar la duración activa.

· Los simulacros seleccionan preguntas de forma aleatoria respetando la distribución por los 7 módulos.

7. Endpoints de la API

Todos los endpoints protegidos requieren token de Laravel Sanctum en el encabezado Authorization: Bearer <token>. Las rutas con rol se filtran mediante el middleware rol.

Método	Endpoint	Acceso	Descripción
GET	/api/ping	Público	Verifica que la API responda.
POST	/api/auth/login	Público	Inicia sesión con correo y contraseña.
POST	/api/auth/registro	Público	Registra un nuevo estudiante.
POST	/api/password/solicitar	Público	Solicita recuperación de contraseña.
GET	/api/password/validar-token	Público	Valida token de recuperación.
POST	/api/password/restablecer	Público	Restablece la contraseña.
POST	/api/auth/logout	Autenticado	Cierra la sesión actual.
GET	/api/auth/me	Autenticado	Devuelve el usuario autenticado.
GET	/api/usuarios/{id}	Autenticado	Obtiene información de un usuario.
PUT	/api/usuarios/{id}	Autenticado	Actualiza perfil propio o usuario permitido.
GET	/api/usuarios	ADMIN	Lista usuarios con filtros.
POST	/api/usuarios	ADMIN	Crea un usuario desde administración.
PATCH	/api/usuarios/{id}/estado	ADMIN	Activa o desactiva usuario.
PATCH	/api/usuarios/{id}/rol	ADMIN	Cambia rol del usuario.
DELETE	/api/usuarios/{id}	ADMIN	Elimina usuario.
GET	/api/usuarios-resumen	ADMIN	Obtiene resumen por rol.

GET	/api/modulos	Autenticado	Lista los 7 módulos temáticos.
GET	/api/modulos/{id}/subtemas	Autenticado	Lista subtemas de un módulo.
GET	/api/diagnostico/estado	ESTUDIANTE	Consulta si el estudiante completó diagnóstico.
POST	/api/diagnostico/iniciar	ESTUDIANTE	Crea intento de diagnóstico.
POST	/api/diagnostico/{intentoId}/responder	ESTUDIANTE	Registra respuesta del diagnóstico.
POST	/api/diagnostico/{intentoId}/finalizar	ESTUDIANTE	Finaliza diagnóstico y calcula resultados.
GET	/api/diagnostico/{intentoId}/resultados	ESTUDIANTE	Consulta resultados del diagnóstico.
GET	/api/ruta	ESTUDIANTE	Obtiene ruta de aprendizaje activa.
POST	/api/plan/generar	ESTUDIANTE	Genera plan de estudio semanal.
GET	/api/plan	ESTUDIANTE	Obtiene plan semanal actual.
GET	/api/ejercicios	TUTOR, ADMIN, REVISOR, ESTUDIANTE	Lista ejercicios según filtros y rol.
GET	/api/ejercicios/{id}	TUTOR, ADMIN, REVISOR, ESTUDIANTE	Obtiene detalle de ejercicio.
POST	/api/ejercicios	TUTOR, ADMIN	Crea nuevo ejercicio.
PUT	/api/ejercicios/{id}	TUTOR, ADMIN	Actualiza ejercicio.
POST	/api/ejercicios/{id}/enviar-revision	TUTOR, ADMIN	Envía ejercicio a revisión.
POST	/api/ejercicios/{id}/aprobar	REVISOR, ADMIN	Aprueba ejercicio.
POST	/api/ejercicios/{id}/rechazar	REVISOR, ADMIN	Rechaza ejercicio.
POST	/api/ejercicios/{id}/publicar	ADMIN	Publica ejercicio aprobado.

POST	/api/ejercicios/{id}/deshabilitar	ADMIN	Deshabilita ejercicio publicado.
GET	/api/practica/activa	ESTUDIANTE	Consulta práctica en proceso.
GET	/api/practica/historial	ESTUDIANTE	Lista prácticas finalizadas del estudiante.
POST	/api/practica/iniciar	ESTUDIANTE	Inicia práctica libre o guiada.
POST	/api/practica/guardados	ESTUDIANTE	Guarda ejercicio para revisar después.
GET	/api/practica/guardados	ESTUDIANTE	Lista ejercicios guardados.
DELETE	/api/practica/guardados/{ejercicioId}	ESTUDIANTE	Quita ejercicio guardado.
GET	/api/practica/{sesionId}	ESTUDIANTE	Consulta detalle de sesión de práctica.
POST	/api/practica/{sesionId}/responder	ESTUDIANTE	Registra respuesta de práctica.
POST	/api/practica/{sesionId}/omitir	ESTUDIANTE	Omite ejercicio por tiempo.
POST	/api/practica/{sesionId}/finalizar	ESTUDIANTE	Finaliza práctica.
GET	/api/practica/{sesionId}/resumen	ESTUDIANTE	Obtiene resumen final de práctica.
GET	/api/simulacros/configuracion	ESTUDIANTE	Consulta configuración activa del simulacro.
GET	/api/simulacros/activa	ESTUDIANTE	Consulta simulacro en proceso.
GET	/api/simulacros/historial	ESTUDIANTE	Lista simulacros anteriores.
POST	/api/simulacros/iniciar	ESTUDIANTE	Inicia simulacro.
GET	/api/simulacros/{simulacroId}	ESTUDIANTE	Consulta detalle del simulacro.
POST	/api/simulacros/{simulacroId}/responder	ESTUDIANTE	Registra respuesta del simulacro.

POST	/api/simulacros/{simulacroId}/finalizar	ESTUDIANTE	Finaliza simulacro y calcula resultados.
GET	/api/admin/simulacros/configuracion	ADMIN	Consulta configuración activa de simulacros.
PUT	/api/admin/simulacros/configuracion	ADMIN	Actualiza duración y puntaje mínimo.
GET	/api/estadisticas/estudiante	ESTUDIANTE	Obtiene dashboard y estadísticas generales del estudiante.
GET	/api/estadisticas/tutor	TUTOR ADMIN	Obtiene dashboard y métricas de estudiantes para el tutor.
GET	/api/estadisticas/tutor	TUTOR ADMIN	Exporta reporte CSV de estudiantes del tutor.
GET	/api/estadisticas/tutor/exportar	ADMIN	Obtiene dashboard administrativo general de la plataforma.
GET	/api/estadisticas/admin	ADMIN	Obtiene dashboard administrativo general de la plataforma.
GET	/api/recursos	ESTUDIANTE, TUTOR, REVISOR, ADMIN	Lista recursos educativos con filtros.
GET	/api/recursos/ejercicio/{ejercicioId}	ESTUDIANTE, TUTOR, REVISOR, ADMIN	Lista recursos vinculados a un ejercicio.
GET	/api/recursos/{id}	ESTUDIANTE, TUTOR, REVISOR, ADMIN	Obtiene detalle de un recurso educativo.
POST	/api/recursos	TUTOR, ADMIN	Crea un nuevo recurso educativo.
PUT	/api/recursos/{id}	TUTOR, ADMIN	Actualiza un recurso educativo.

DELETE	/api/recursos/{id}	TUTOR, ADMIN	Elimina un recurso educativo.
PATCH	/api/recursos/{id}/enviar-revision	TUTOR, ADMIN	Envía recurso a revisión.
POST	/api/recursos/{id}/vincular-ejercicio	TUTOR, ADMIN	Adjunta o desadjunta un recurso a un ejercicio.
PATCH	/api/recursos/{id}/aprobar	REVISOR, ADMIN	Aprueba un recurso educativo.
PATCH	/api/recursos/{id}/rechazar	REVISOR, ADMIN	Rechaza un recurso educativo.

Cuerpos de petición principales

POST /api/auth/login

```
{
  "correo": "student@ingeniamath.gt",
  "password": "password"
}
```

POST /api/practica/iniciar

```
{
  "modo": "LIBRE",
  "modulo_id": 2,
  "subtema_id": 5,
  "nivel_dificultad": "BASICO"
}
```

POST /api/practica/{sesionId}/responder

```
{
  "ejercicio_id": 10,
  "opcion_id": 25,
  "respuesta_texto": null,
  "marcado_guardado": false,
  "tiempo_segundos": 45
}
```

PUT /api/admin/simulacros/configuracion

```
{
  "nombre": "Simulacro ingreso USAC",
  "duracion_minutos": 90,
  "puntaje_minimo_aprobacion": 61
}
```

8. Validaciones, seguridad e integridad

Elemento	Implementación
Autenticación	Laravel Sanctum genera y valida tokens para rutas protegidas.
Autorización	ProtectedRoute en frontend y middleware rol en backend limitan acceso por ADMIN, TUTOR, REVISOR y ESTUDIANTE.
Contraseñas	Se almacenan como hash seguro, no como texto plano.
Validación de entrada	FormRequest y Request::validate validan correos, contraseñas, tipos de ejercicios, niveles y cantidades.
Integridad referencial	PostgreSQL aplica llaves foráneas entre usuarios, roles, módulos, ejercicios, prácticas y simulacros.
Restricciones	CHECK constraints controlan estados, niveles, porcentajes, cantidades y rangos permitidos.

9. Instalación y despliegue

Requisitos previos

- PHP 8.2 o superior.
- Composer.
- Node.js y npm.
- PostgreSQL instalado y accesible.
- Git para clonar el repositorio.

Configuración de base de datos

```
psql -U postgres -f DB/script_v2.sql
psql -U adminMath -d ingeniaMathDB -f DB/inserts_completos_v2.sql
```

Variables .env del backend

```
APP_NAME=IngeniaMath
APP_ENV=local
APP_KEY=base64:...
```

```
APP_DEBUG=true
APP_URL=http://localhost:8000

DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=ingeniaMathDB
DB_USERNAME=adminMath
DB_PASSWORD=admin123
DB_SCHEMA=math
```

Ejecución del backend

```
cd backend
composer install
php artisan key:generate
php artisan config:clear
php artisan serve --host=0.0.0.0 --port=8000
```

Ejecución del frontend

```
cd frontend
npm install
npm run dev -- --host 0.0.0.0
```

Para pruebas desde celular, ambos servidores deben escuchar en 0.0.0.0 y el dispositivo debe estar conectado a la misma red WiFi. El navegador del celular debe ingresar a la IP local de la computadora y al puerto configurado por Vite.

10. Responsividad y Pruebas en Dispositivos Móviles

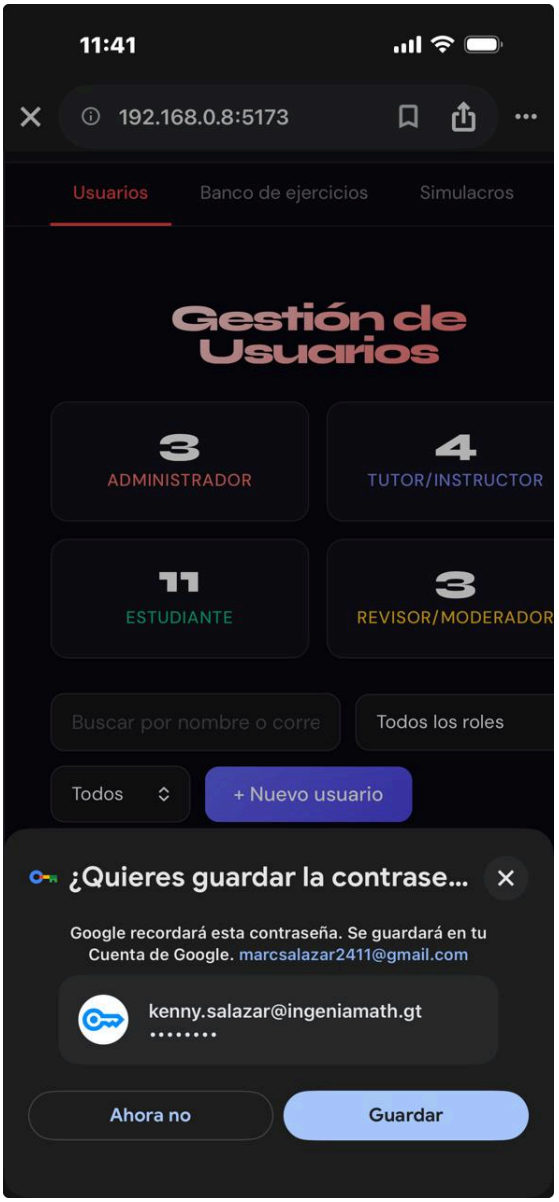
Para el desarrollo del sistema se configuró el entorno para que pudiera ser accedido desde otros dispositivos dentro de la misma red local. Para esto, el backend en Laravel se ejecuto utilizando el comando php artisan serve --host=0.0.0.0, y el frontend con Vite configurado tambien con host: '0.0.0.0', lo cual permite que la aplicacion sea accesible mediante la direccion IP local del equipo.

Se realizaron pruebas desde un telefono movil conectado a la misma red WiFi, accediendo a la aplicacion mediante la direccion 192.168.0.8:5173. Desde este dispositivo se verifico que

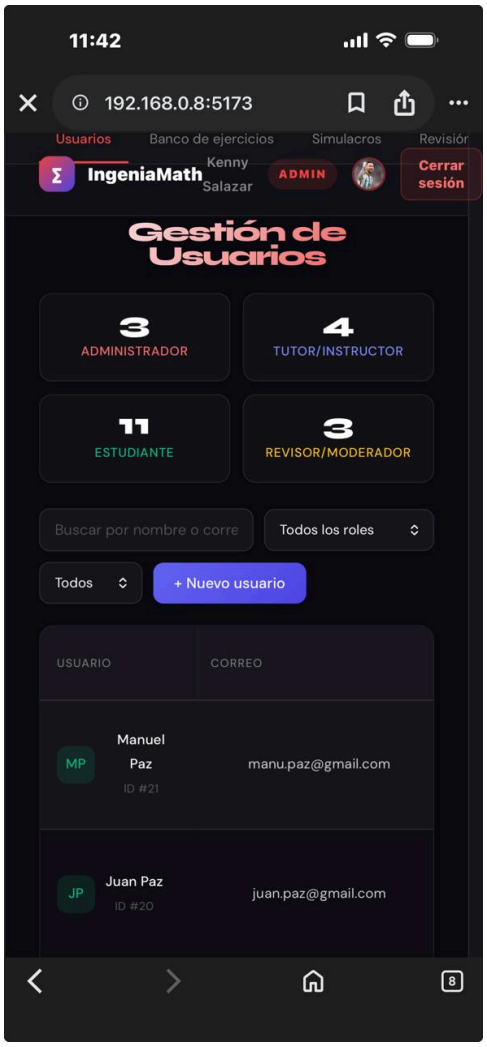
todas las funcionalidades principales trabajaran correctamente, incluyendo inicio de sesion, registro de usuario, recuperacion de contraseña y gestion de usuarios.

En el caso de la recuperacion de contraseña, el flujo funciona de la siguiente manera: el usuario ingresa a la opcion de “olvide mi contraseña”, introduce su correo electronico y recibe un enlace de recuperacion. Dicho enlace redirige a una ruta dentro del sistema con un token incluido, por ejemplo 192.168.0.8:5173/recuperar-password?token=.... Para que este proceso funcione correctamente, el dispositivo movil debe estar conectado a la misma red WiFi donde se encuentra corriendo el servidor.

Este comportamiento cumple con lo solicitado en el enunciado del proyecto, ya que el sistema está diseñado para funcionar en un entorno de red local. En un entorno real de produccion se utilizaria un dominio en lugar de una direccion IP, pero para fines academicos esta implementación es válida y suficiente.

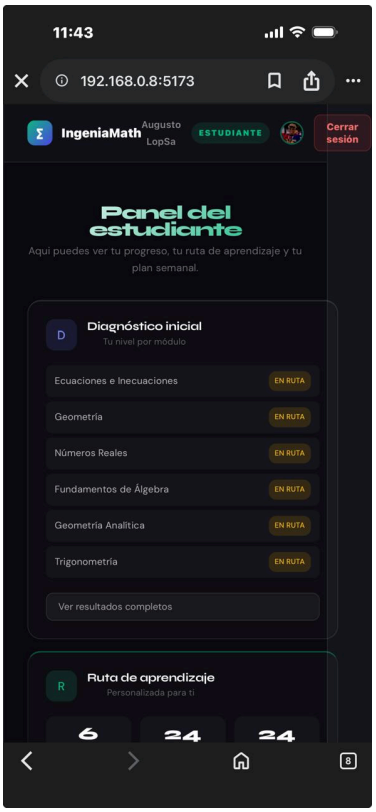


Inicio de sesiones con un admin

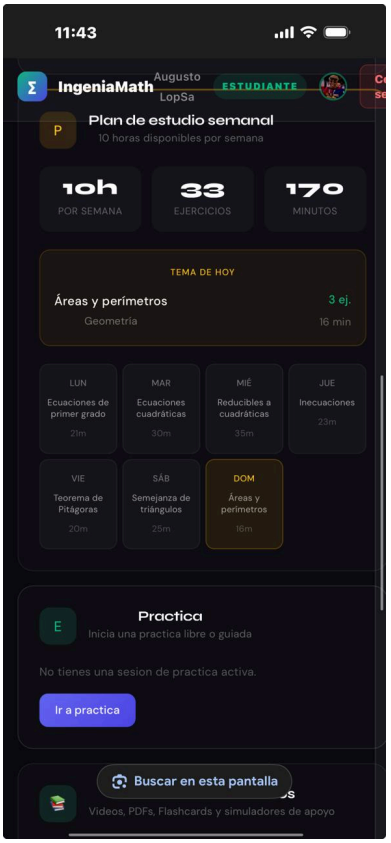

11:42
192.168.0.8:5173
Usuarios
Banco de ejercicios
Simulacros
Revisión
IngeniaMath
Kenny Salazar
ADMIN
Cerrar sesión
Gestión de Usuarios
3
ADMINISTRADOR
4
TUTOR/INSTRUCTOR
11
ESTUDIANTE
3
REVISOR/MODERADOR
Buscar por nombre o corre
Todos los roles
Todos
+ Nuevo usuario
USUARIO
CORREO
MP
Manuel Paz
ID #21
manu.paz@gmail.com
JP
Juan Paz
ID #20
juan.paz@gmail.com

11:42
192.168.0.8:5173
Nuevo usuario
Nombres
Apellidos
Correo electrónico
Contraseña
Rol
Estudiante
Cancelar
Crear usuario

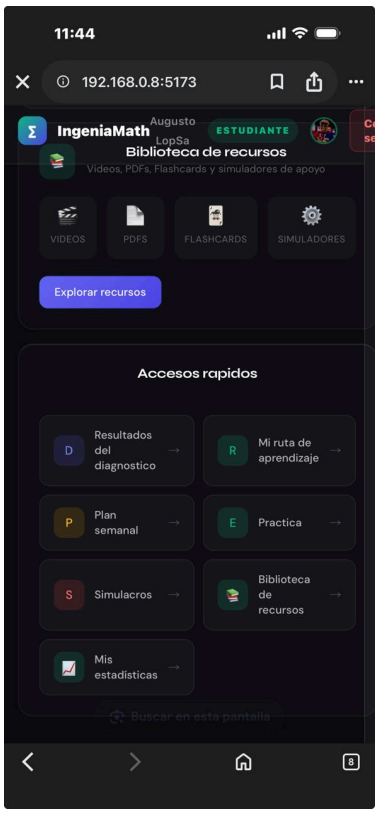
Creación de nuevo usuario

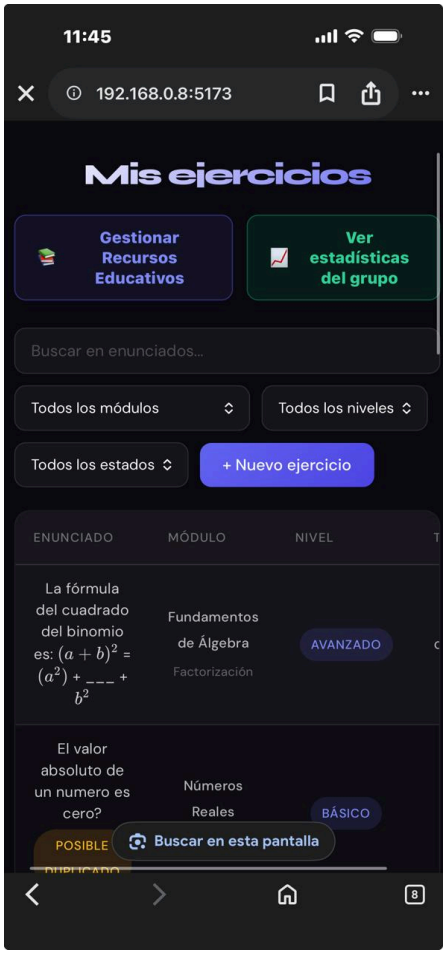


Inicio de sesión con un estudiante

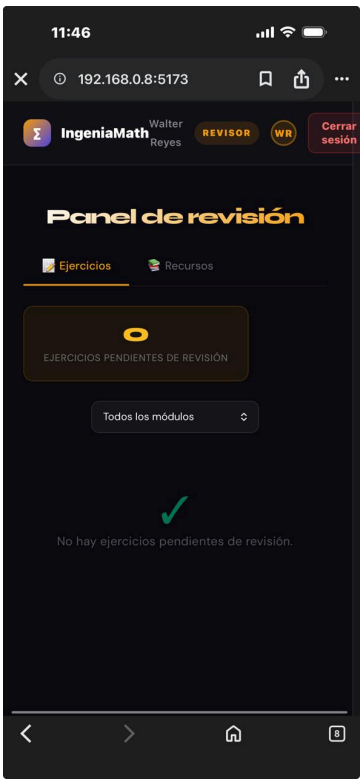


Estudiante





Inicio de sesión con un tutor



Inicio de sesión con un revisor

11. Diagramas del sistema

Diagrama ER

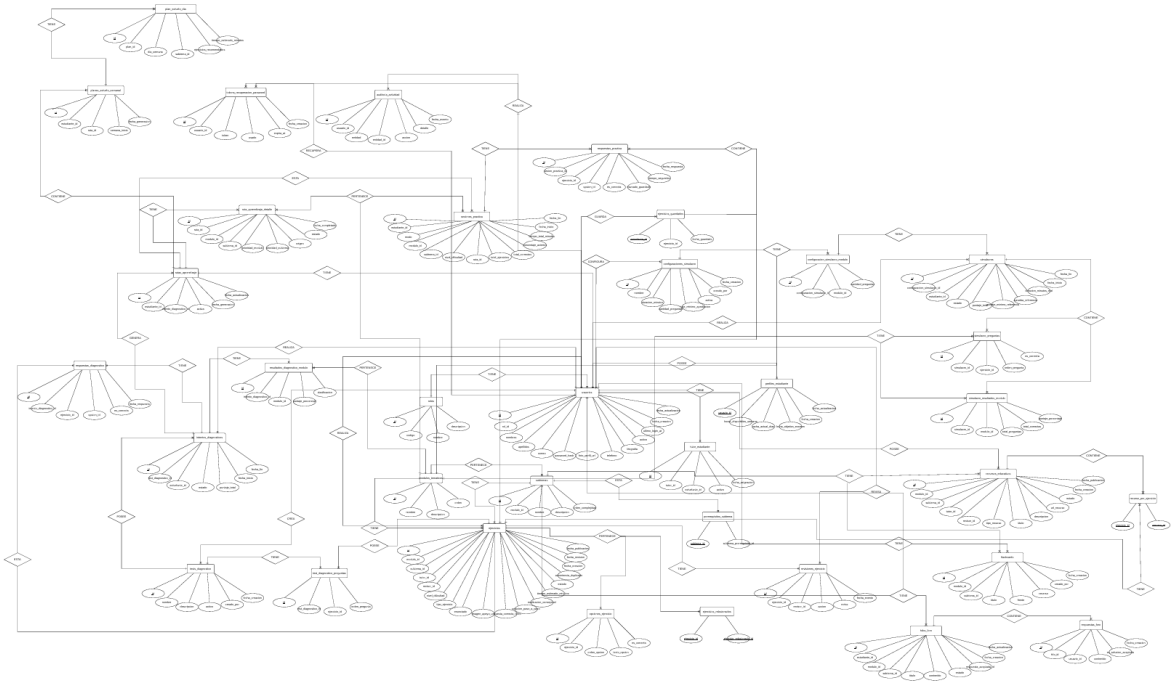


Diagrama Relacional

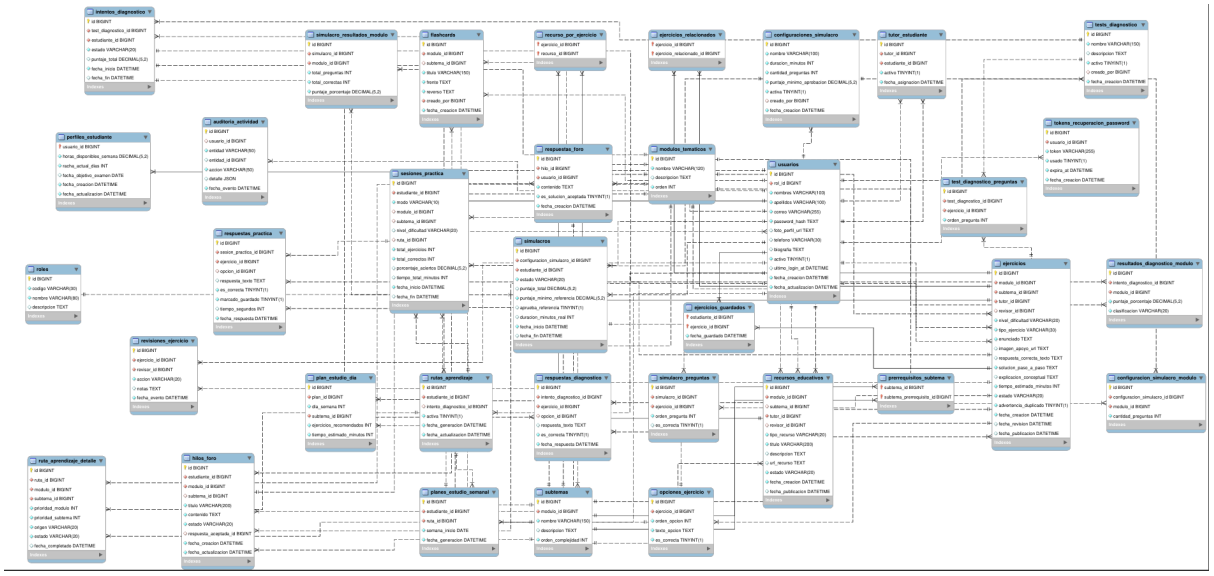


Diagrama de Componentes

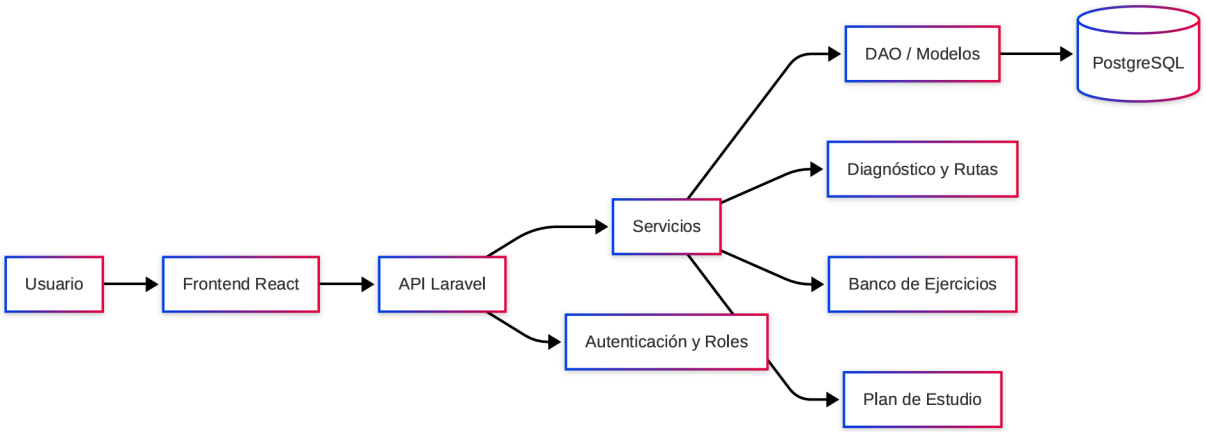


Diagrama de Paquetes

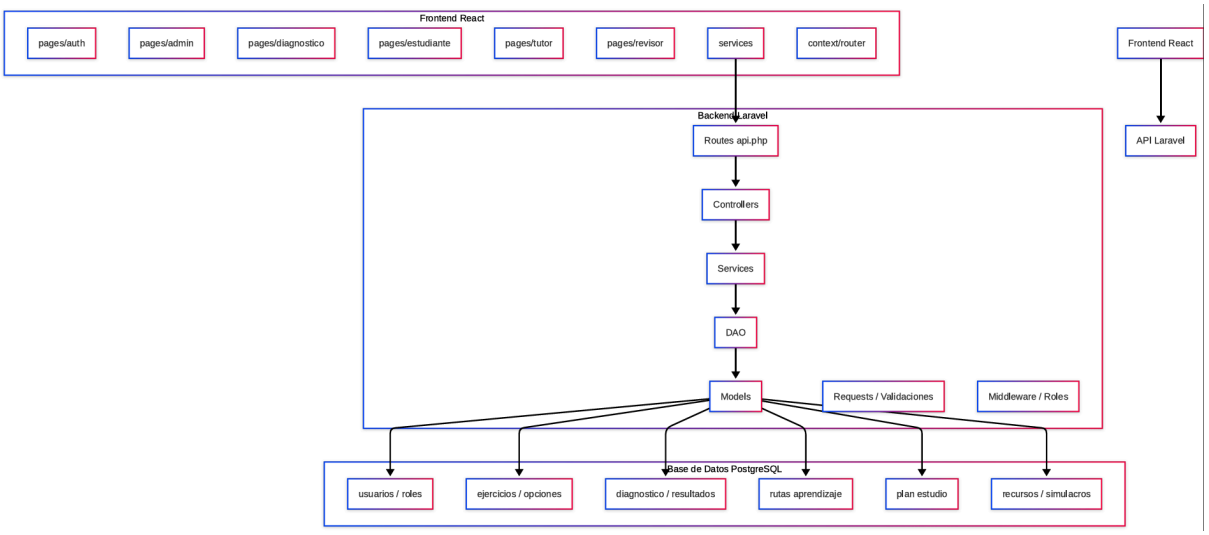
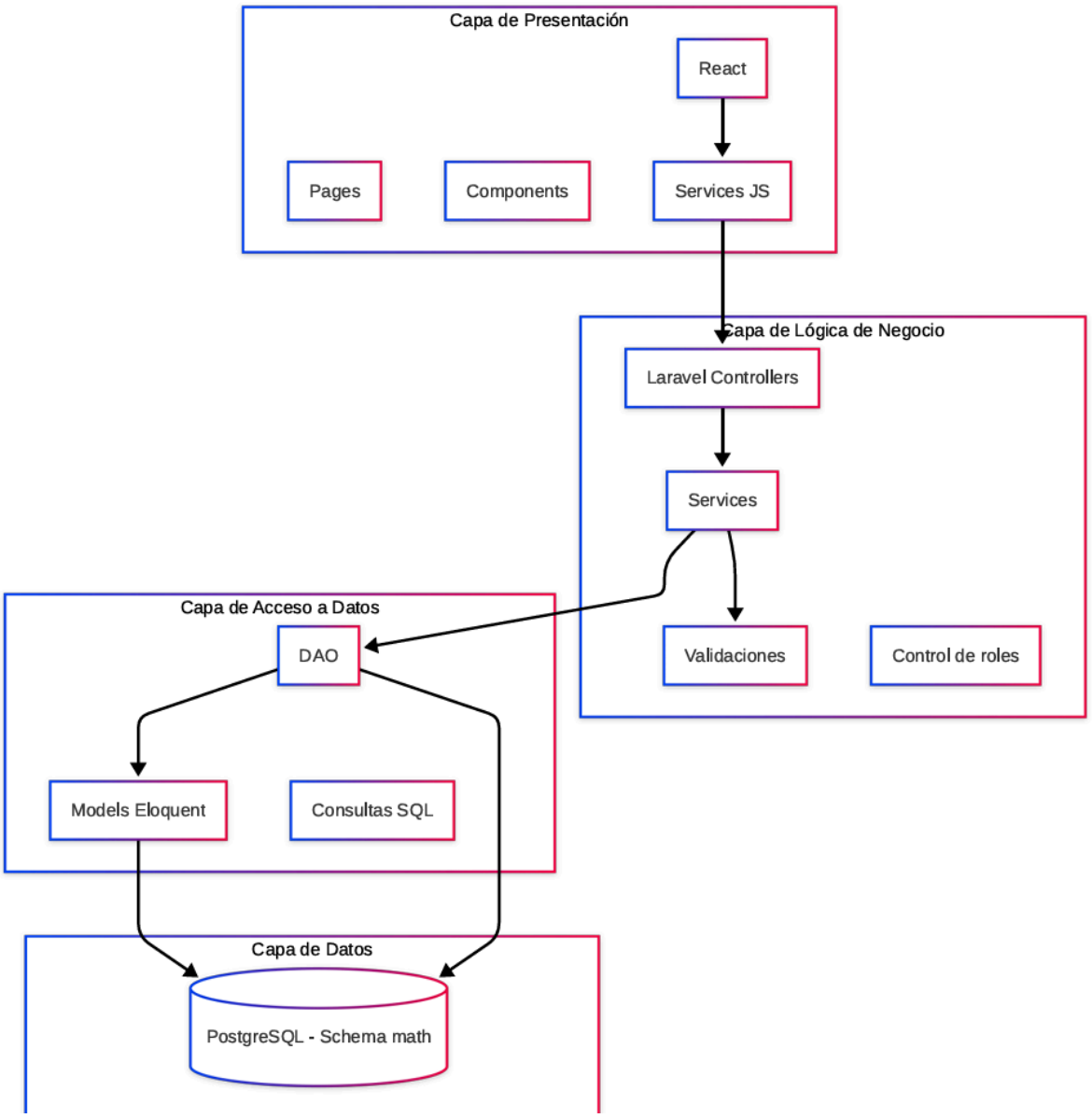


Diagrama N Capas


Capa de Presentación
React
Pages
Components
Services JS
Capa de Lógica de Negocio
Laravel Controllers
Services
Validaciones
Control de roles
Capa de Acceso a Datos
DAO
Models Eloquent
Consultas SQL
Capa de Datos
PostgreSQL - Schema math